

Concurrency and Distributed Systems

Week 7 – Part 3: Fan-Out / Fan-In Async Design

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 3 Roadmap

1 Pattern

2 Implementation

3 Demo

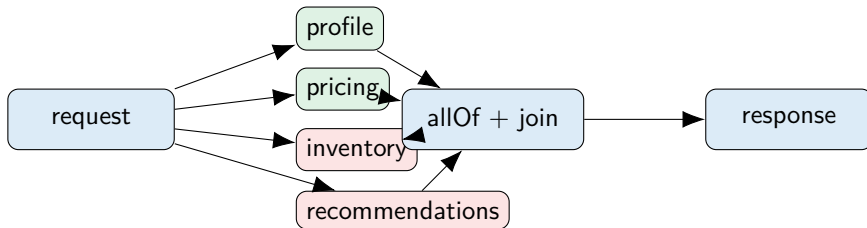
Fan-out

Start independent service calls at roughly the same time.

Fan-in

Wait for the required branches, aggregate their results, and expose partial failure if fallback happened.

Service Composition Shape



Fan-Out Branches

```
1 CompletableFuture<ServiceResult> profile =  
2     service("profile", "ok", fast, false, executor);  
3  
4 CompletableFuture<ServiceResult> inventory =  
5     service("inventory", "slow", slow, false, executor)  
6         .completeOnTimeout(  
7             fallback("inventory"),  
8             timeout.toMillis(),  
9             TimeUnit.MILLISECONDS);
```

Fan-In Aggregation

```
1 return CompletableFuture.allOf(profile, pricing, inventory, recs)
2   .thenApply(ignored -> {
3       List<ServiceResult> results = Stream
4         .of(profile, pricing, inventory, recs)
5         .map(CompletableFuture::join)
6         .toList();
7
8       return new CombinedResponse(results, usedFallback(results));
9   });
```

Design Rule

Do not aggregate invisible failure

If a branch times out or fails, the combined response must expose that fallback was used.

Week 7 response model

Each `ServiceResult` has a fallback flag. The combined response has a global fallback flag.

Run

```
.\run-demo.ps1 06
```

- Identify the four service branches.
- Identify the slow branch and timeout fallback.
- Identify the failing branch and exception fallback.
- Identify the fan-in line.

Exercise 4

Implement `Ex4_FanOutFanInPipeline` so the tests see four results and fallback for inventory and recommendations.

- Use `supplyAsync`.
- Use `allOf`.
- Use explicit fallback values.